

Introducción a la Transformada Wavelet

Multirate , bancos de filtros y aplicaciones biomédicas

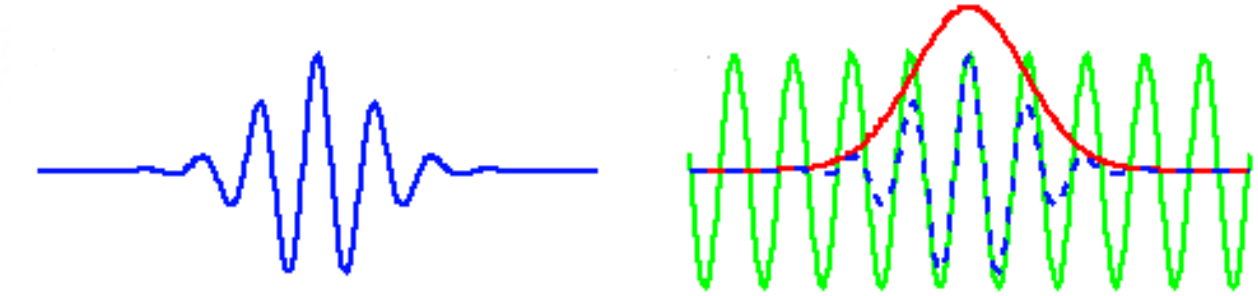
Mag. Ing. CIP Moises Meza Rodriguez

¿Qué es una Wavelet?

Una wavelet es una ondicula que se utiliza para analizar señales a múltiples escalas.

Características:

- Duración finita
- Localización temporal
- Multiescala



A diferencia de Fourier:

- No usa senos infinitos
- Conserva información temporal

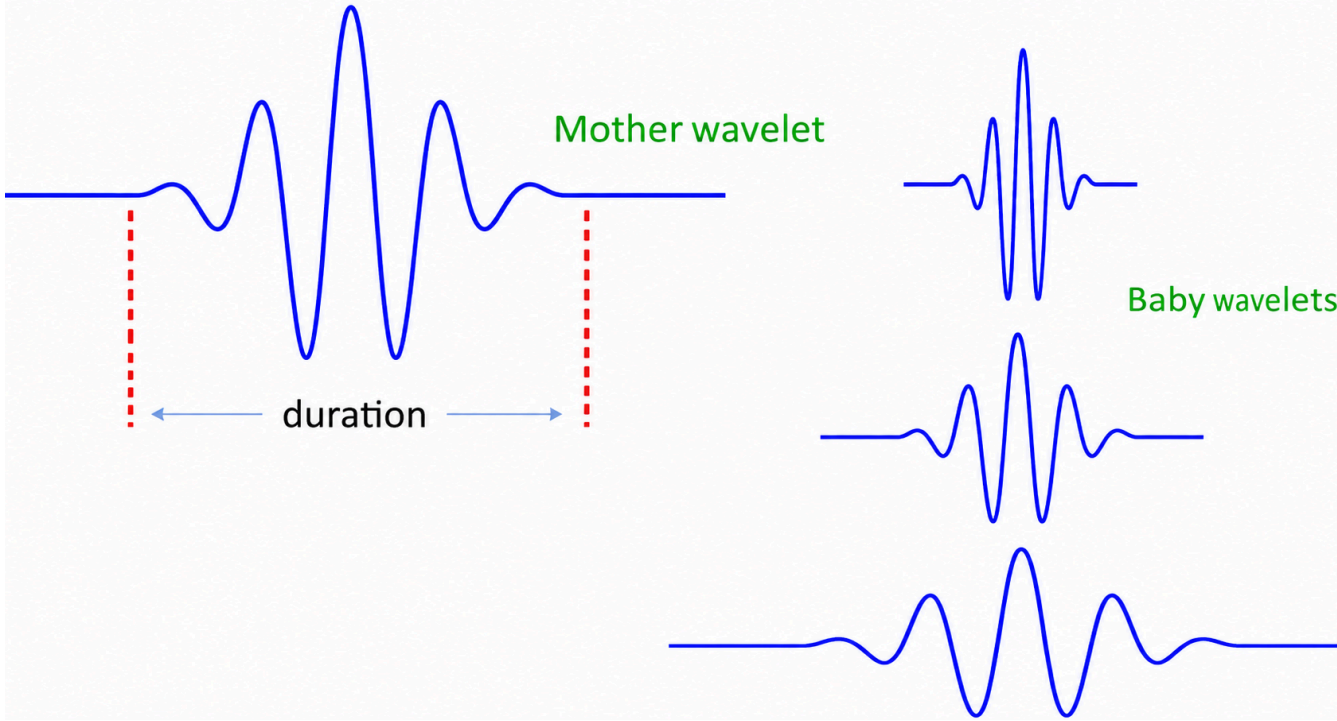
Wavelet madre

La wavelet base se representa como: $\psi(t)$

Las demás wavelets se obtienen mediante:

- Escalamiento
- Traslación

$$\psi_{a,b}(t) = \frac{1}{\sqrt{|a|}}\psi\left(\frac{t-b}{a}\right)$$



Escala y frecuencia

Escala pequeña

- Wavelet comprimida
- Altas frecuencias
- Eventos rápidos

Escala grande

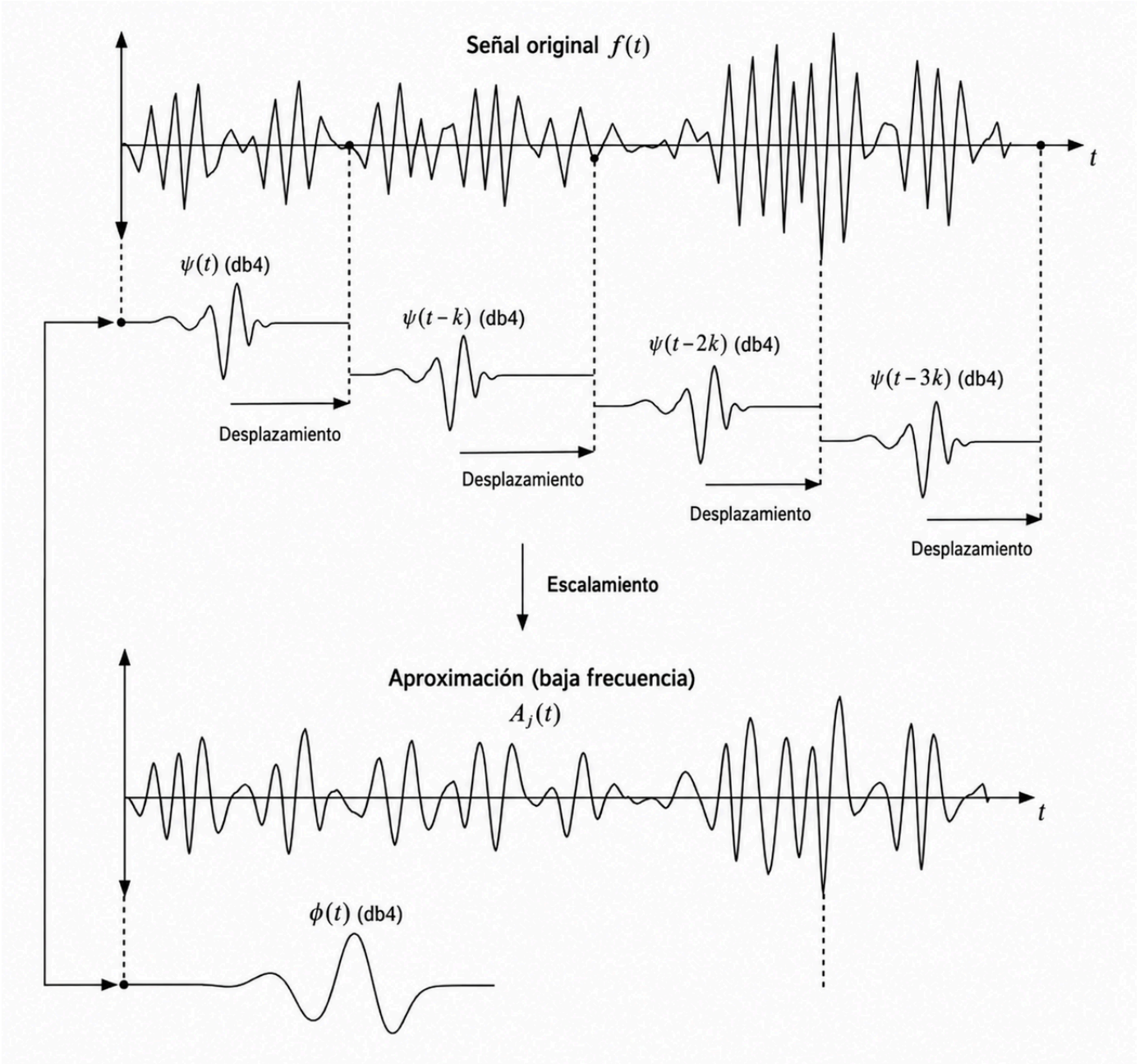
- Wavelet expandida
- Bajas frecuencias
- Tendencias lentas



¿Por qué estudiar Wavelets?

La transformada wavelet permite analizar señales:

- No estacionarias
- Transitorias
- Multiescala
- Tiempo–frecuencia



Scalogram

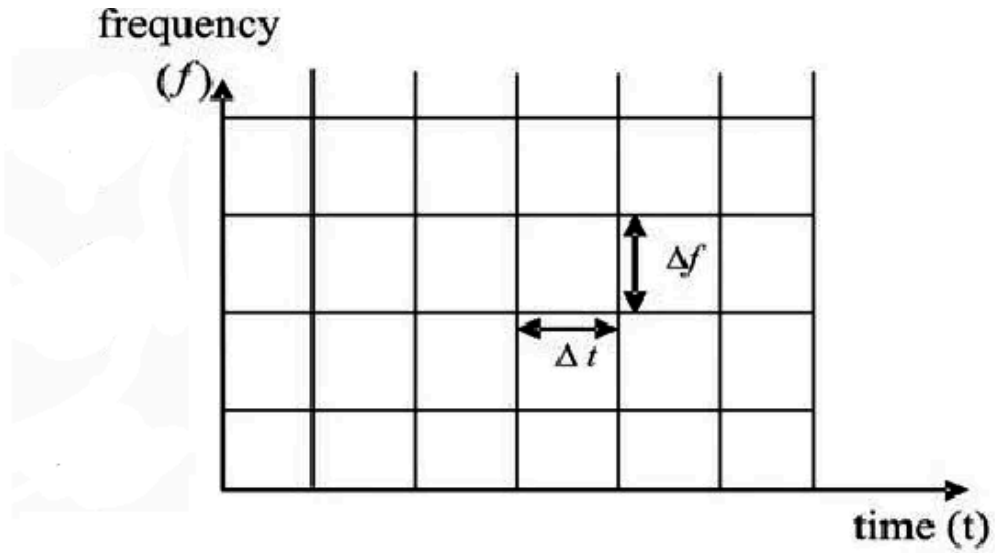
Representación tiempo–frecuencia.

Ejes:

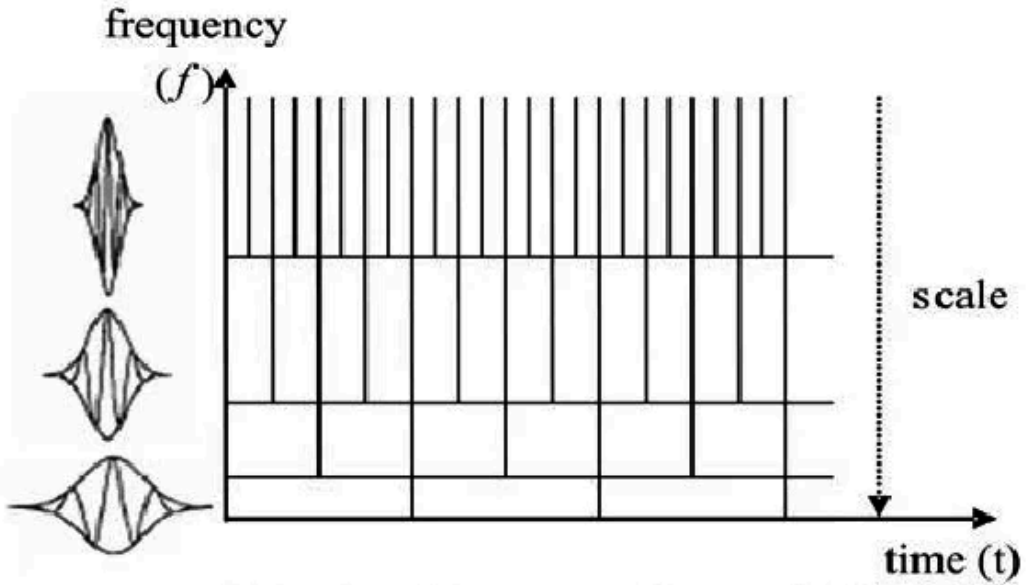
- X → tiempo
- Y → escala/frecuencia
- Color → energía

Muy usado en:

- EEG
- Audio
- Sismología



The time-frequency window of STFT



The time-frequency window of Wavelet

Transformada Wavelet Continua

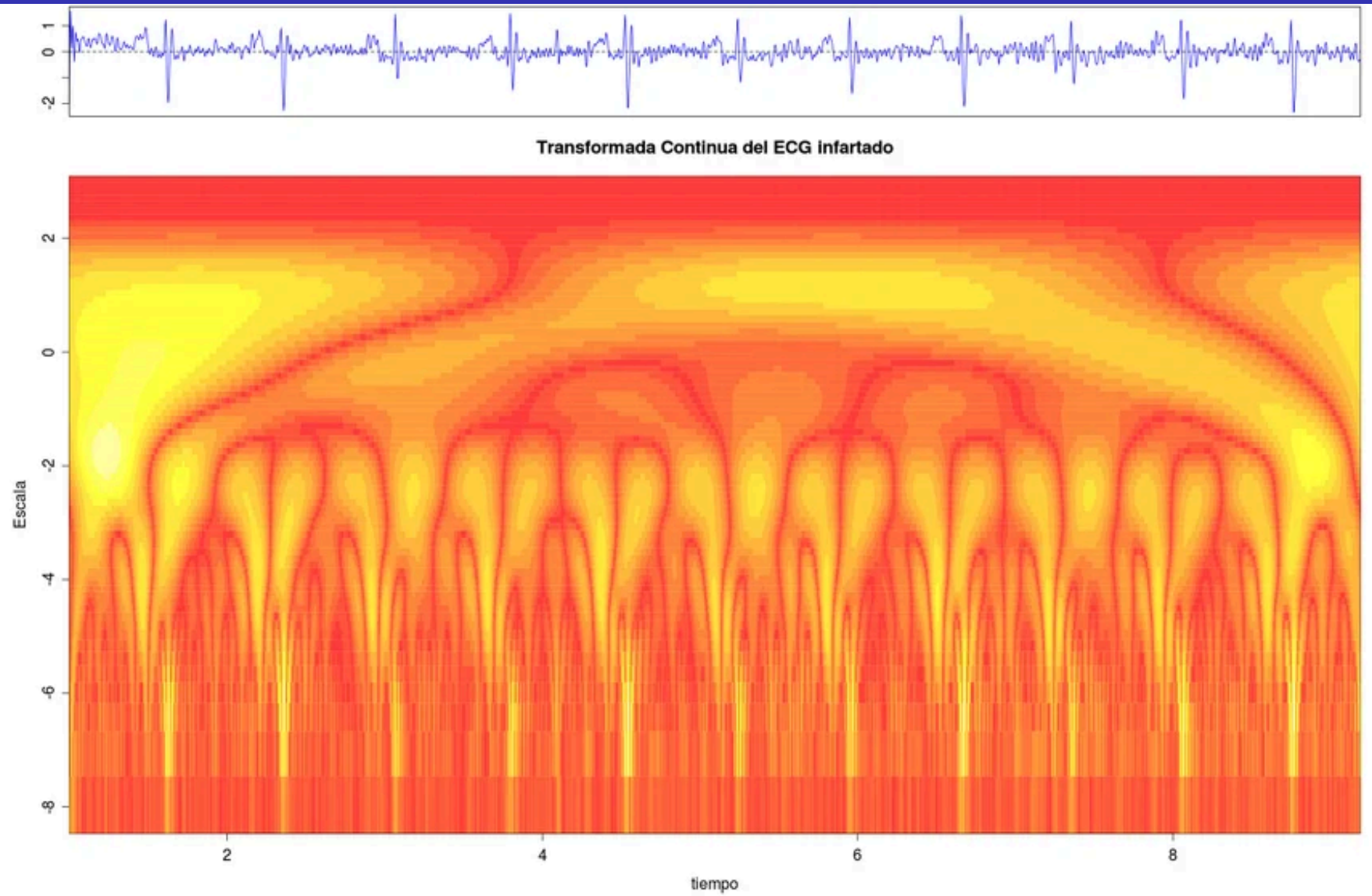
$$W(a,b) = \frac{1}{\sqrt{|a|}} \int_{-\infty}^{\infty} x(t)\psi^* \left(\frac{t-b}{a}\right) dt$$

Idea intuitiva

La wavelet se desplaza sobre la señal preguntando:

> “¿La señal se parece a mí en este instante y esta escala?”

Transformada Wavelet Continua



PyWavelets - Wavelet Transforms in Python

Contents

Installing
API Reference
Usage examples
Contributing
Development guide
Release Notes

PyWavelets is open source wavelet transform software for Python. It combines a simple high level interface with low level C and Cython performance.

PyWavelets is very easy to use and get started with. Just install the package, open the Python interactive shell and type:

```
import pywt
cA, cD = pywt.dwt([1, 2, 3, 4], 'db1')
```

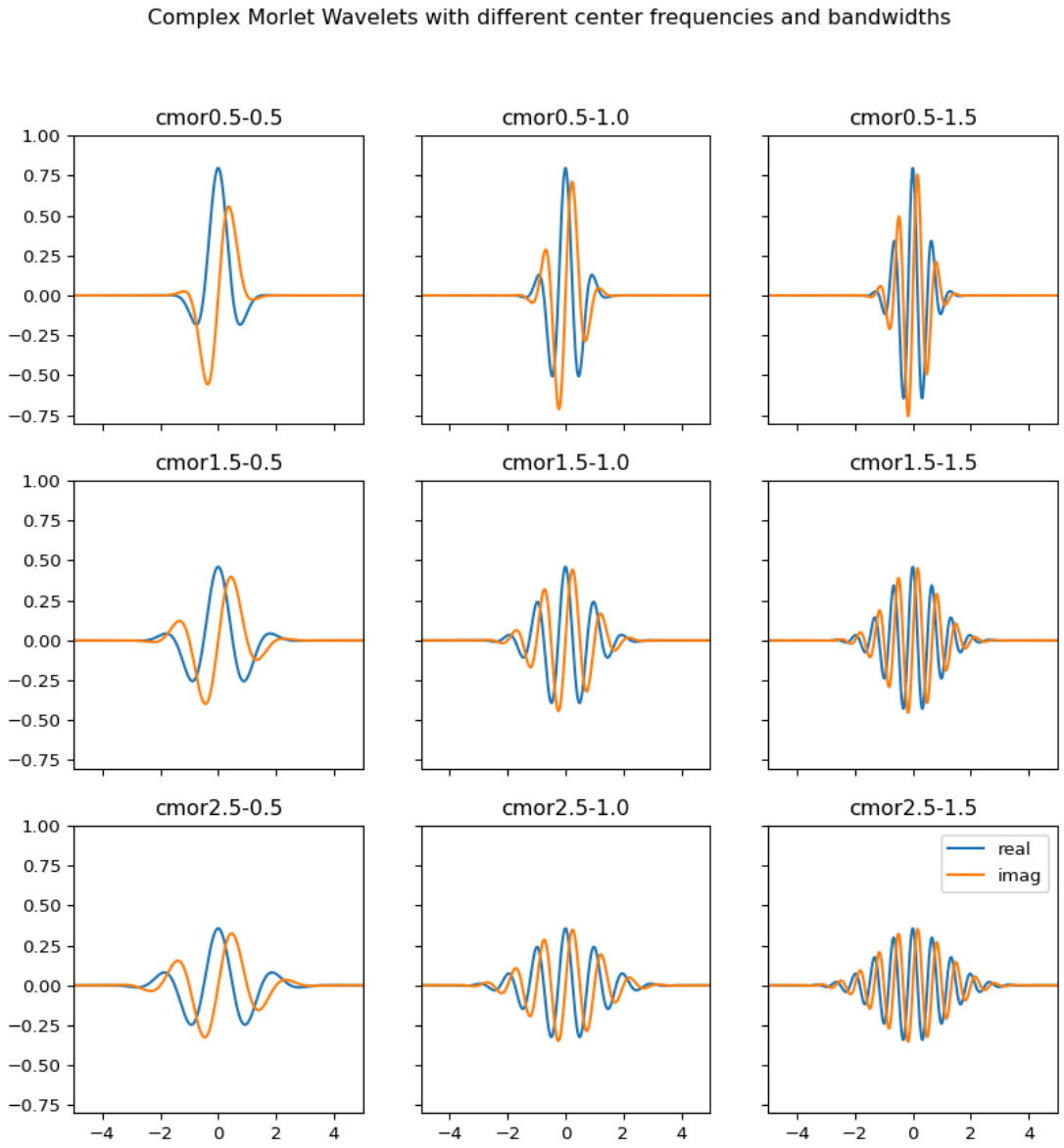
On this page
Contents
Main features
Getting help
License
Citing

Transformada Wavelet Continua

```
!pip install pywavelets
```

```
import pywt
pywt.families()
#pywt.wavelist(kind='continuous')
```

['haar', 'db', 'sym', 'coif', 'bior', 'rbio', 'dmey', 'gaus', 'mexh', 'morl', 'cgau', 'shan', 'fbsp', 'cmor']



Transformada Wavelet Discreta

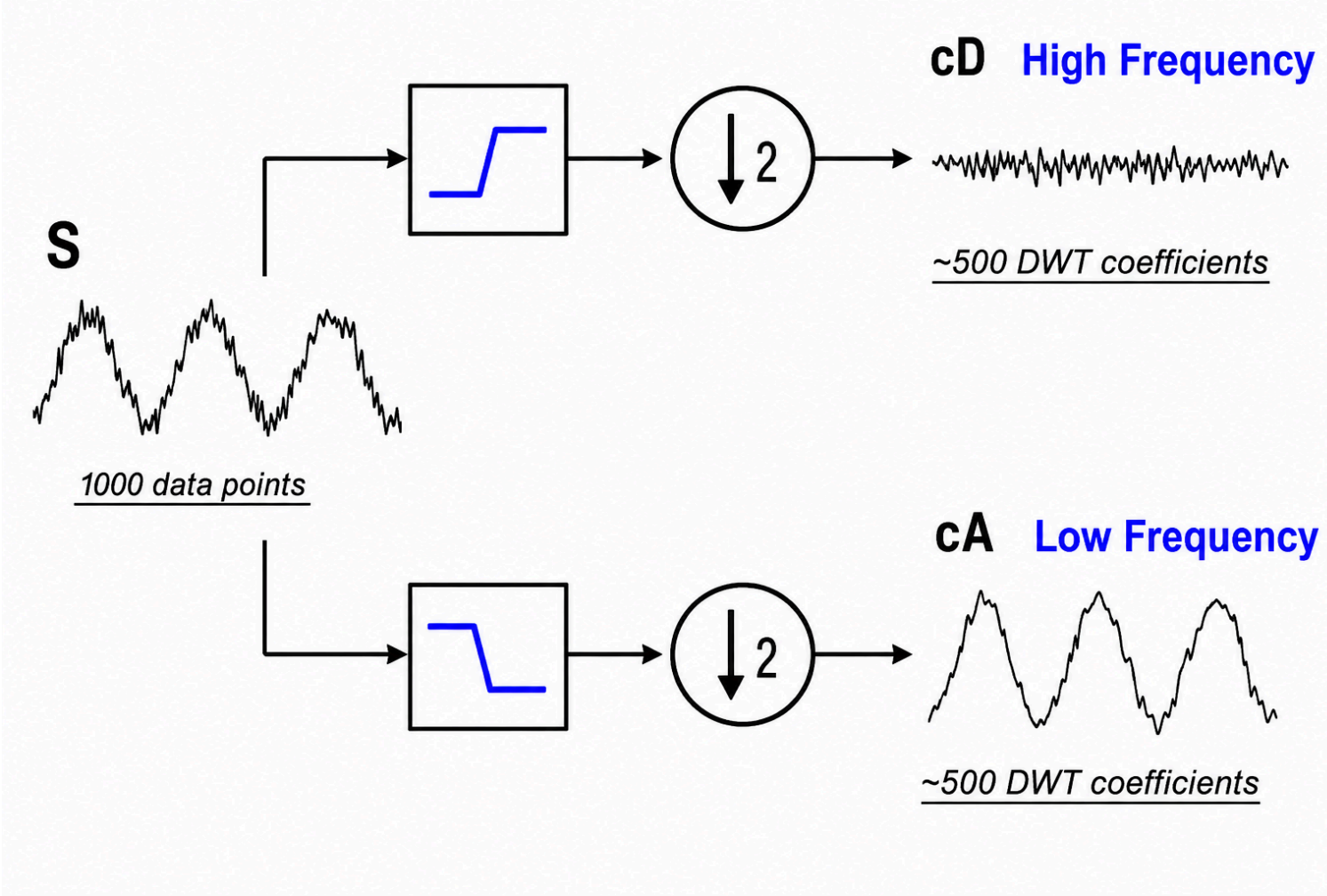
La DWT utiliza:

- Bancos de filtros
- Decimación
- Multiresolución

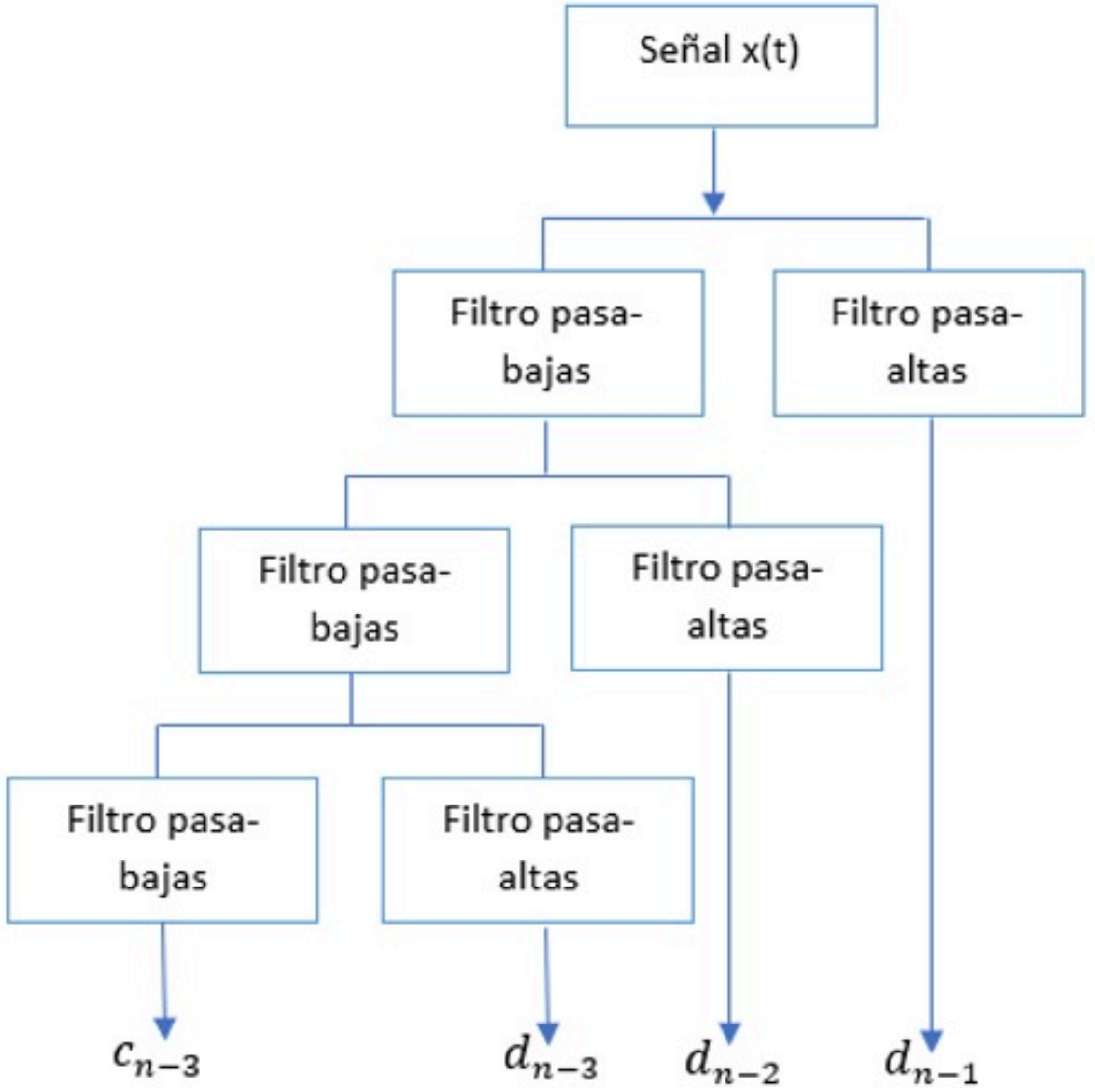
Produce:

- Aproximaciones
- Detalles

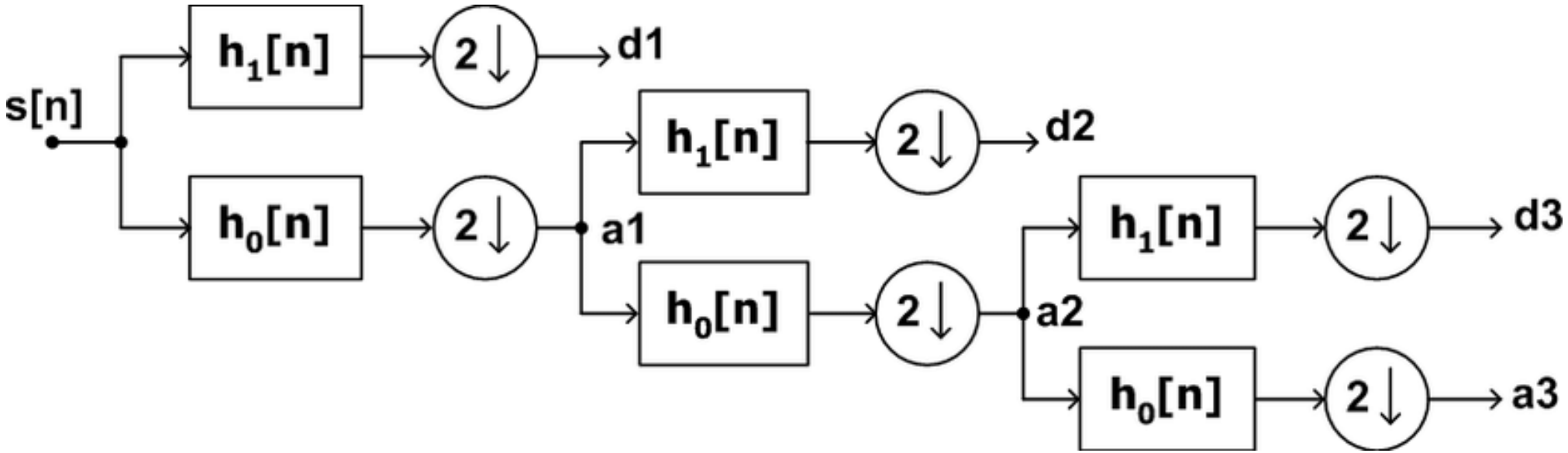
Es una de las técnicas más utilizada en DSP.



Transformada Wavelet Discreta



Transformada Wavelet Discreta



Decimación (Downsampling)

La decimación reduce la frecuencia de muestreo.

$$y[n] = x[Mn]$$

fs/2

Si:

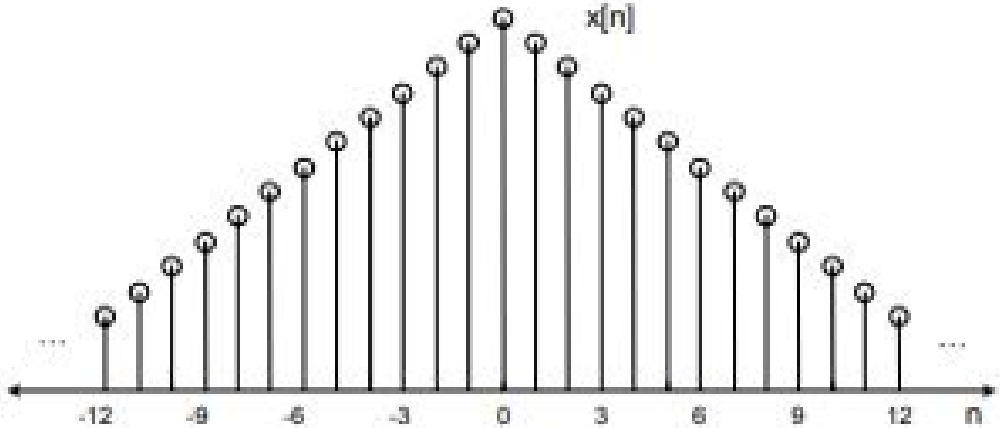
$$M = 3$$

Entonces se conserva:

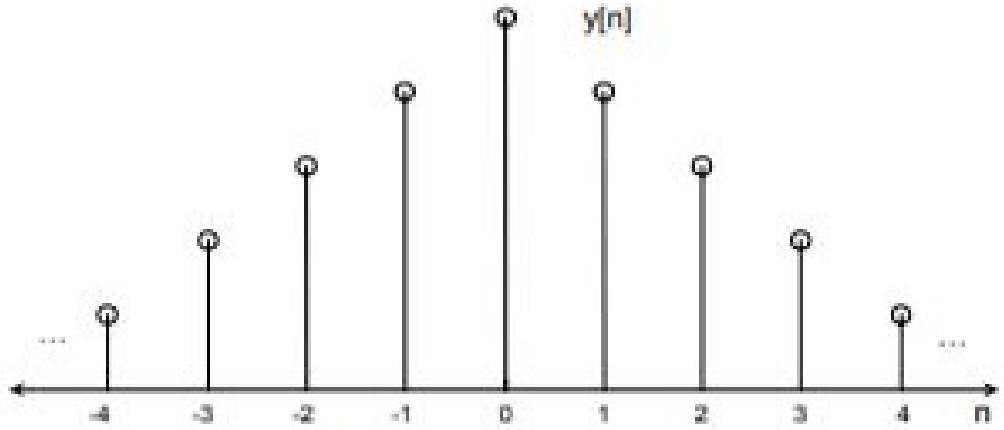
- Tres muestras de "x" serian una de "y"

Ejemplo:

$$[1, 2, 3, 4, 5, 6, 7] \rightarrow [1, 4, 7]$$

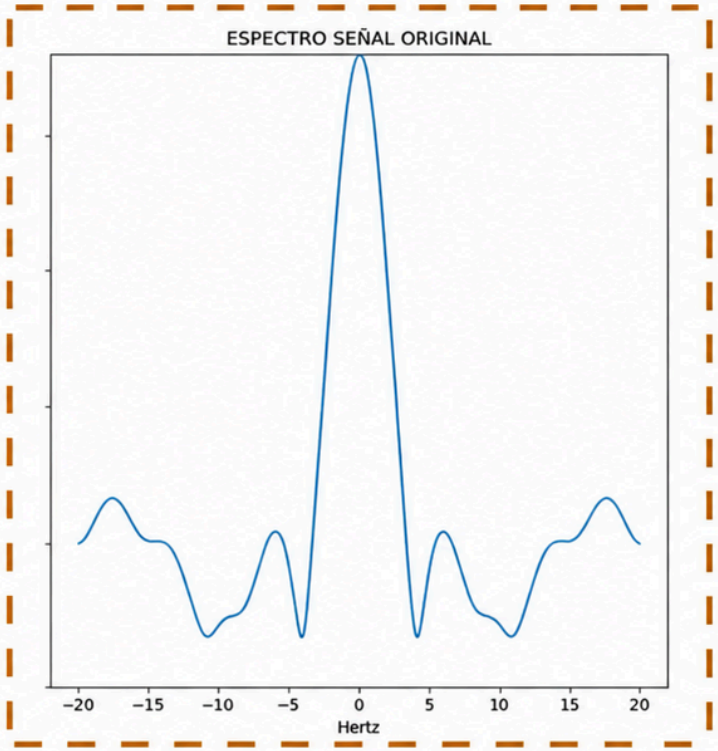


Orignal Discrete Signal



Decimation of a discrete-time signal by a factor of 3

X[F]

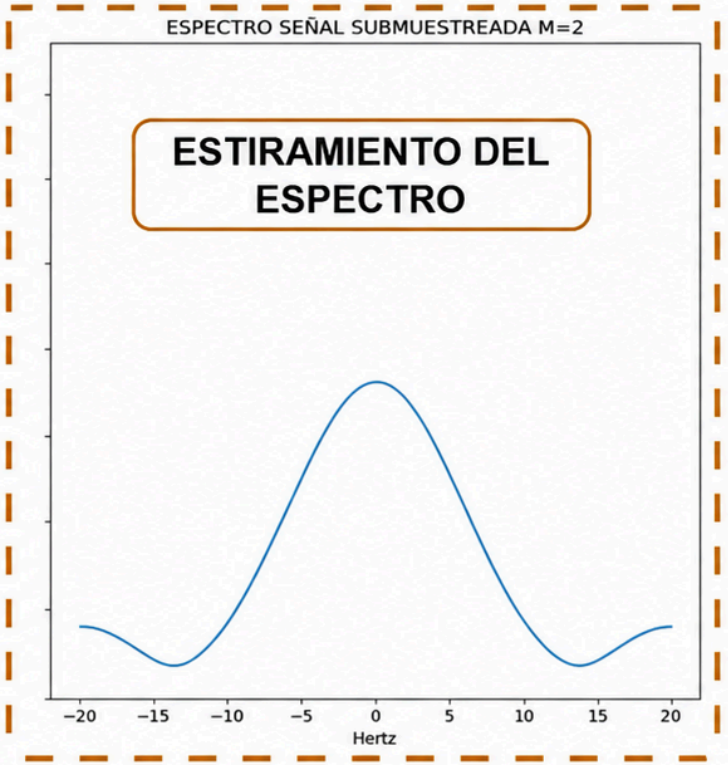


$$y_D(n) = x(Mn)$$

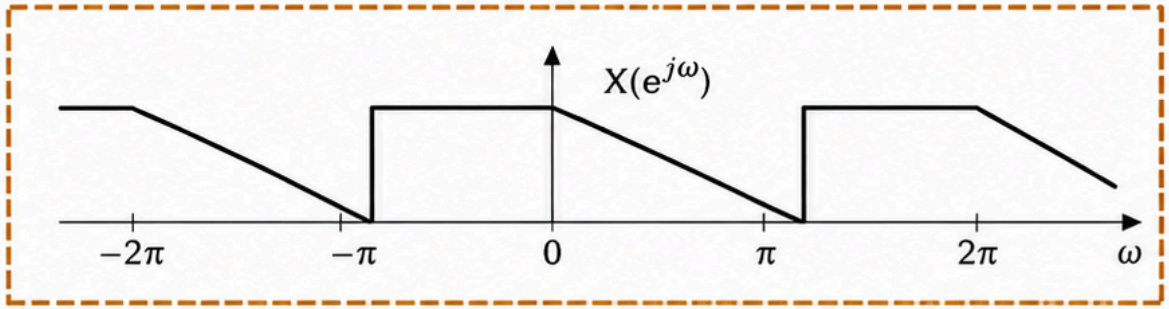
M=2

↓M

YD[F]



SEÑAL ORIGINAL

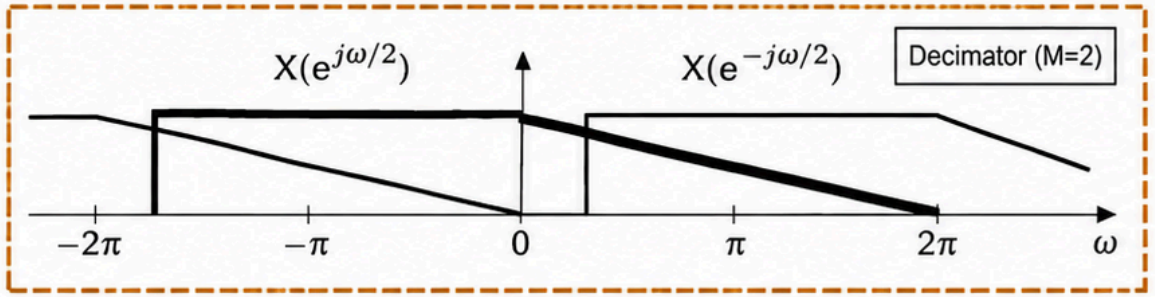


$$y_D(n) = x(Mn).$$

$$Y_D(z) = \frac{1}{M}\sum_{k=0}^{M-1} X(z^{1/M}W^k).$$

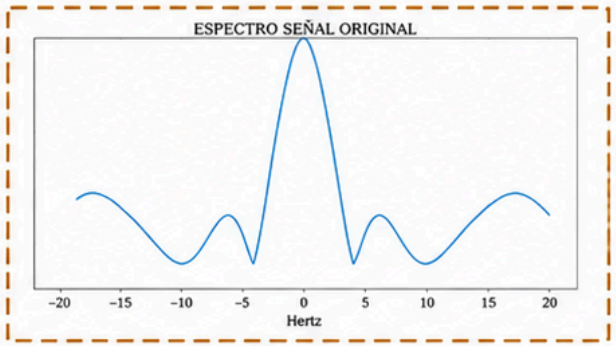
ESTIRAMIENTO

SEÑAL SUB-MUESTREADA



$$z = e^{jw}$$

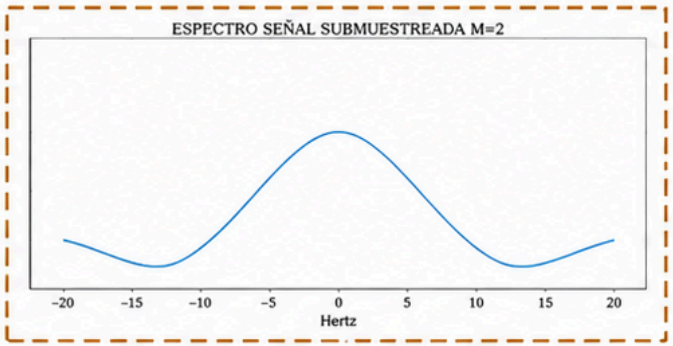
X[F]



M=2



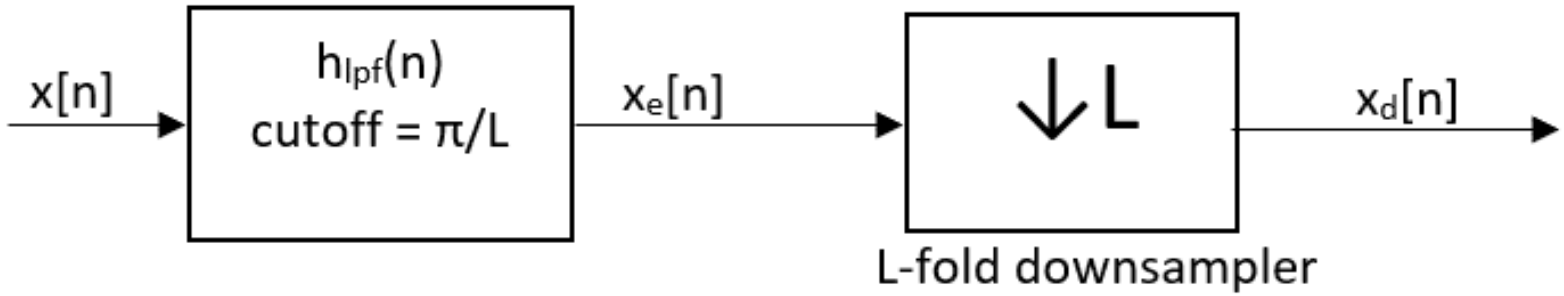
YD[F]



Aliasing

La decimación directa produce:

- Superposición espectral
- Distorsión
- Pérdida de información
- Por ello antes de decimar se utiliza Filtro pasa bajos antialiasing



Interpolación (Upsampling)

La interpolación aumenta la frecuencia de muestreo.

Proceso:

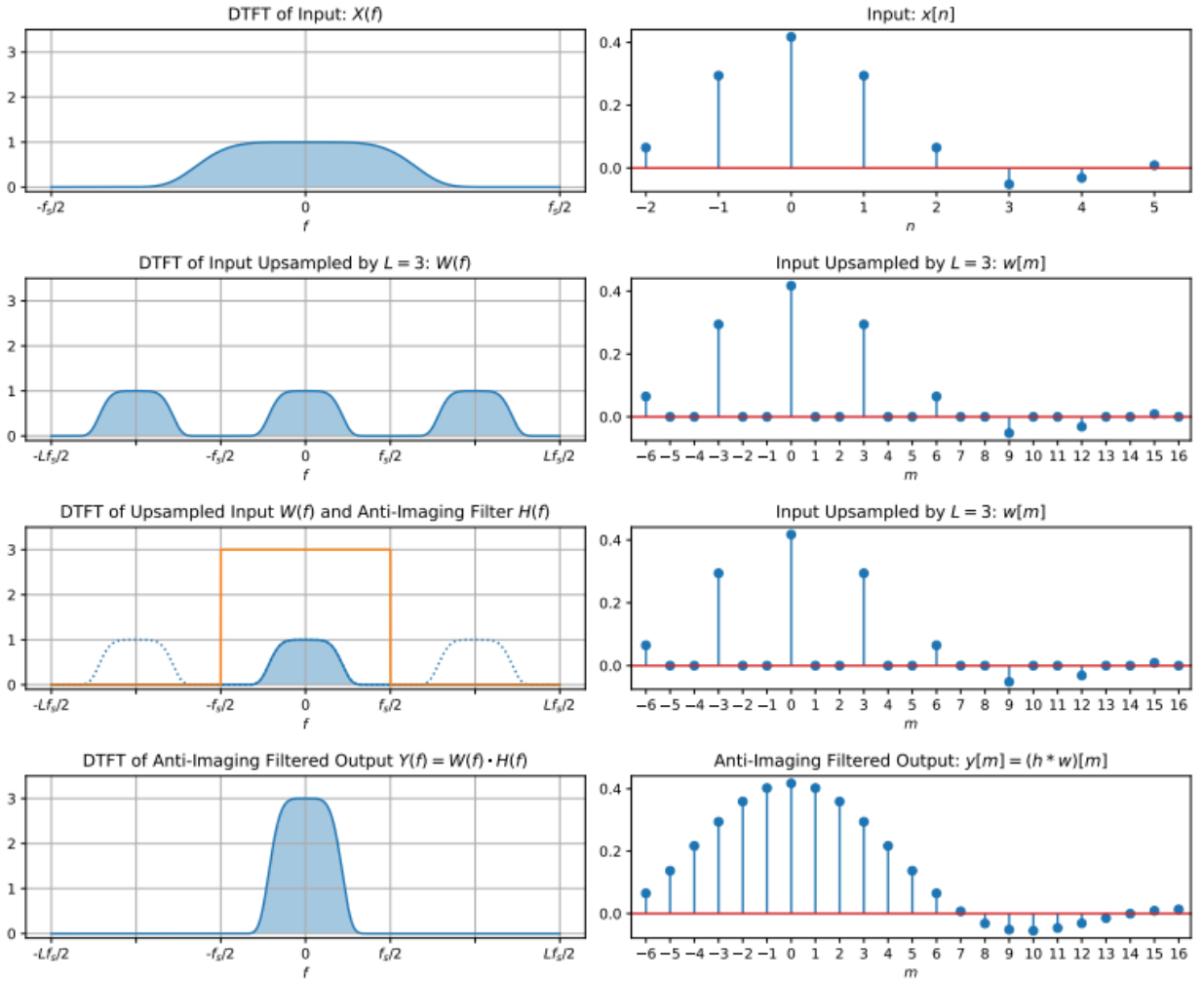
1. Insertar ceros

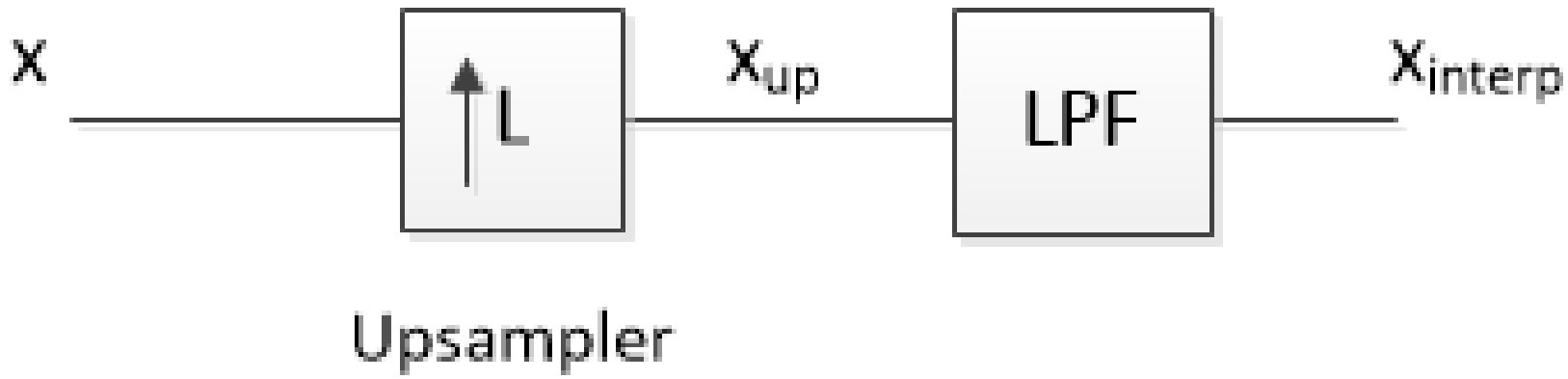
2. Filtrar

$$y[n] = \begin{cases} x[n/L] & n = kL \\ \text{otro caso} \end{cases}$$

Ejemplo:

$$[1, 2, 3] \rightarrow [1, 0, 2, 0, 3, 0]$$




x
L
x_{up}
LPF
x_{interp}
Upsampler

Banco de filtros

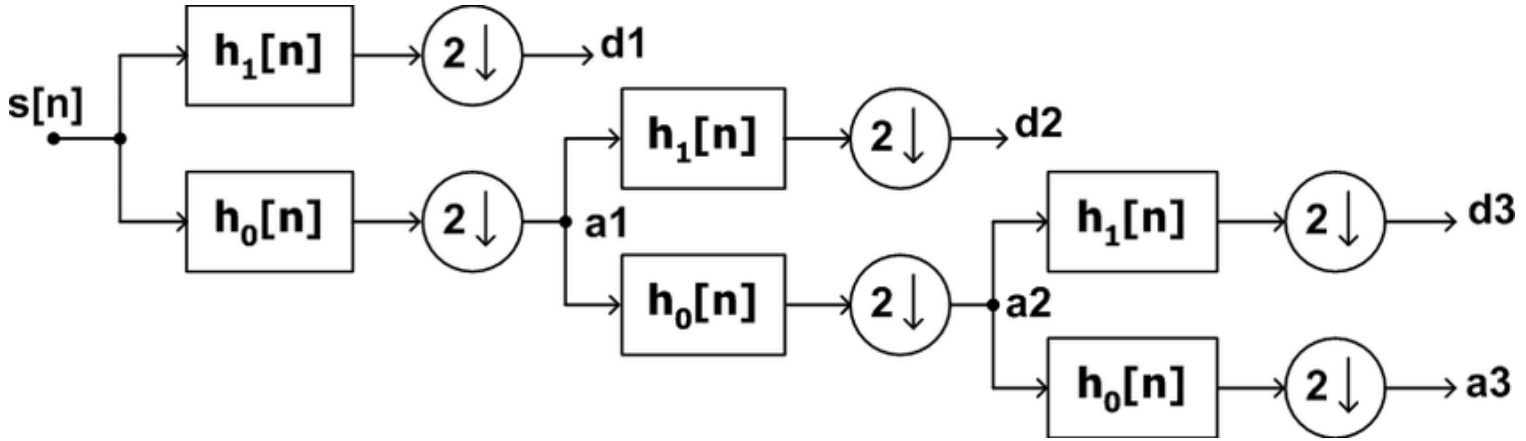
Un banco de filtros divide la señal en subbandas frecuenciales.

Componentes

- LPF → bajas frecuencias
- HPF → altas frecuencias

Resultado

- Aproximación
- Detalle



Aproximaciones y detalles

Componente	Contenido
Aproximación	?
Detalle	?

Detalles contienen

- Bordes
- Transitorios
- Ruido
- Eventos rápidos

Multilevel decomposition using `wavedec`

`pywt.wavedec(data, wavelet, mode='symmetric', level=None, axis=-1)`

Multilevel 1D Discrete Wavelet Transform of data.

Parameters:

data: array_like

Input data

wavelet : *Wavelet object or name string*

Wavelet to use

mode : *str, optional*

Signal extension mode, see Modes.

level : *int, optional*

Decomposition level (must be >= 0). If level is None (default) then it will be calculated using the `dwt_max_level` function.

axis: int, optional

Axis over which to compute the DWT. If not given, the last axis is used.

Returns:

[cA_n, cD_n, cD_n-1, ..., cD2, cD1] : *list*

Ordered list of coefficients arrays where `n` denotes the level of decomposition. The first element (`cA_n`) of the result is approximation coefficients array and the following elements (`cD_n` - `cD_1`) are details coefficients arrays.

Examples

Try it in your browser!

```
>>> from pywt import wavedec
>>> coeffs = wavedec([1,2,3,4,5,6,7,8], 'db1', level=2)
>>> cA2, cD2, cD1 = coeffs
>>> cD1
array([-0.70710678, -0.70710678, -0.70710678, -0.70710678])
>>> cD2
array([-2., -2.])
>>> cA2
array([  5.,  13.])
```

Multilevel reconstruction using `waverec`

pywt.waverec(*coeffs*, *wavelet*, *mode='symmetric'*, *axis=-1*)

Multilevel 1D Inverse Discrete Wavelet Transform.

Parameters:

coeffs : *array_like*

Coefficients list [cAn, cDn, cDn-1, ..., cD2, cD1]

wavelet : *Wavelet object or name string*

Wavelet to use

mode : *str, optional*

Signal extension mode, see Modes.

axis: int, optional

Axis over which to compute the inverse DWT. If not given, the last axis is used.

Notes

It may sometimes be desired to run `waverec` with some sets of coefficients omitted. This can best be done by setting the corresponding arrays to zero arrays of matching shape and dtype. Explicitly removing list entries or setting them to None is not supported.

Specifically, to ignore detail coefficients at level 2, one could do:

```
coeffs[-2] = np.zeros_like(coeffs[-2])
```

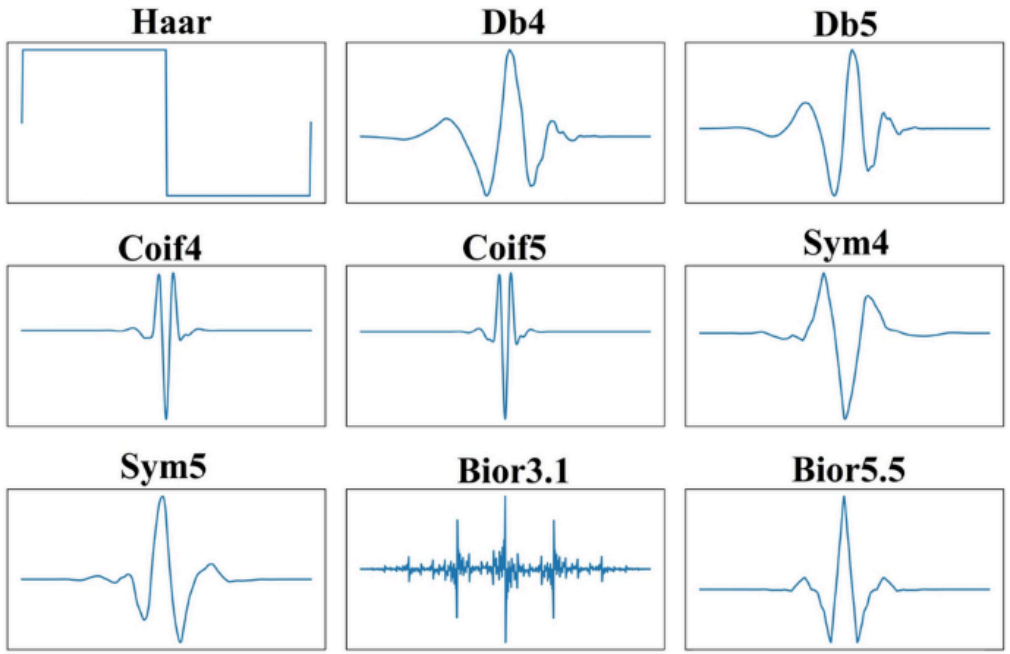
Examples

Try it in your browser!

```
>>> import pywt
>>> coeffs = pywt.wavedec([1,2,3,4,5,6,7,8], 'db1', level=2)
>>> pywt.waverec(coeffs, 'db1')
array([ 1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.])
```

Familias de Wavelets

Familia	Característica
Haar	Muy simple
Daubechies	Muy usada en ECG
Symlets	Más simétrica
Coiflets	Más momentos nulos
Morlet	Excelente para CWT

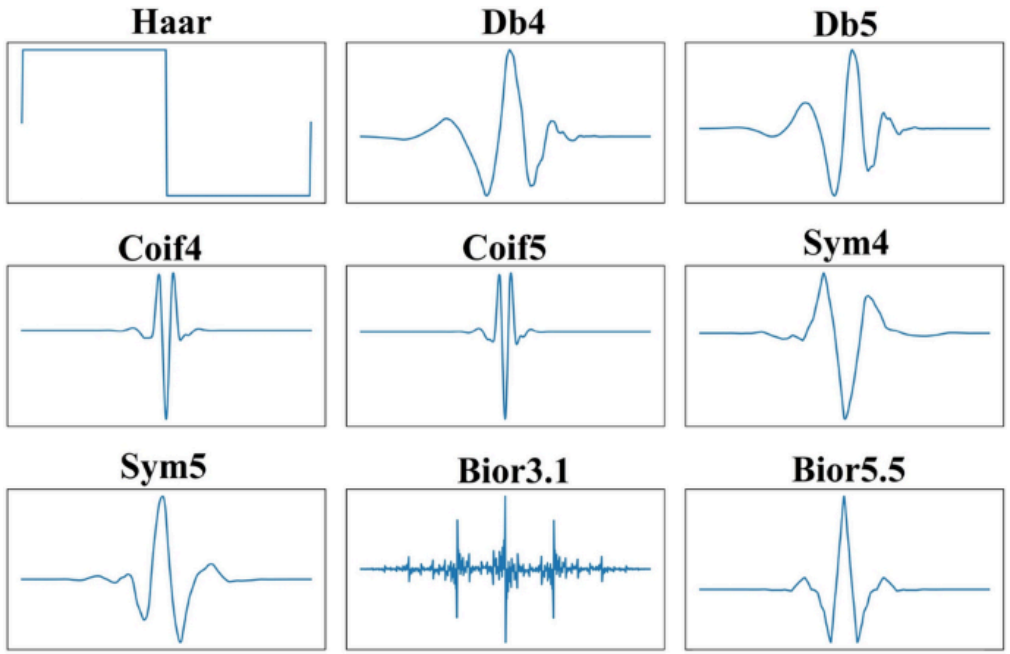


Wavelet Haar

$$\psi(t) = \begin{cases} 1 & 0 \leq t < \frac{1}{2} \\ -1 & \frac{1}{2} \leq t < 1 \\ 0 & \text{otro caso} \end{cases}$$

Características

- Muy rápida
- Discontinua
- Fácil implementación



Aplicaciones biomédicas

ECG

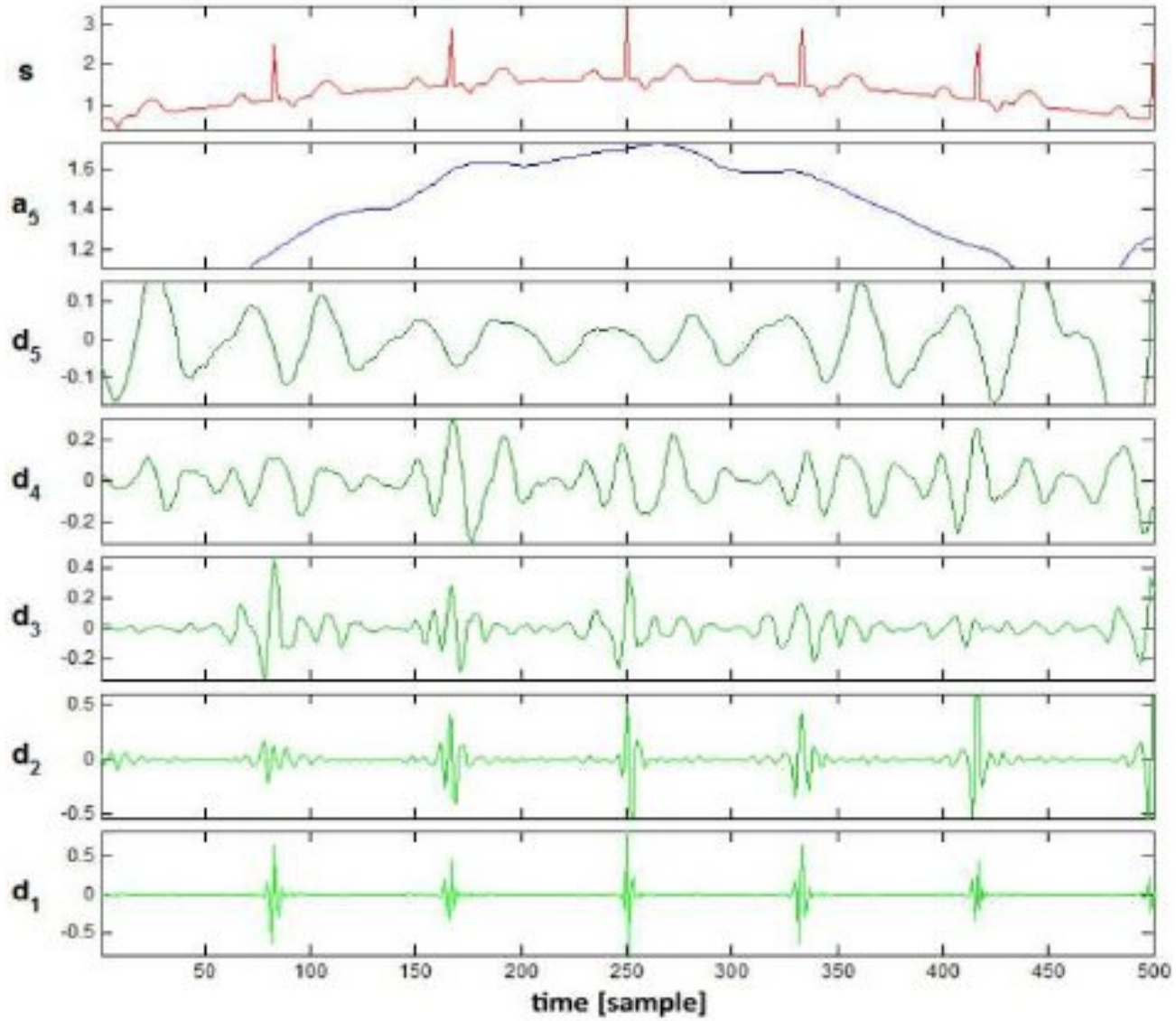
- Detección QRS
- HRV

EEG

- Epilepsia
- Bandas cerebrales
- Artefactos

EMG

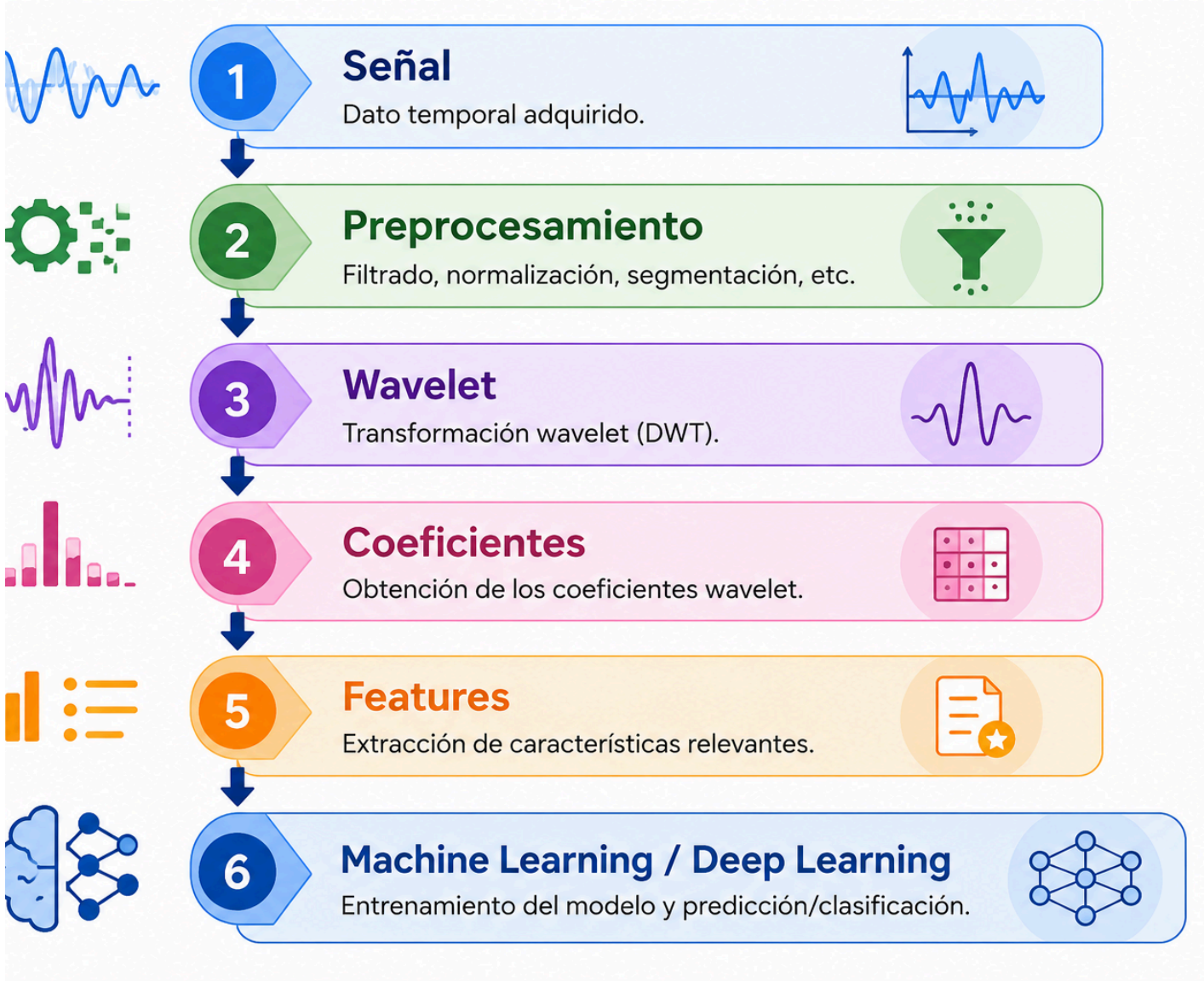
- Fatiga muscular
- Activación muscular



Pipeline de procesamiento

Puntos importantes

- Elegir bien los niveles para denoising
- Elegir bien una wavelet madre segun papers
- Aplicar threshold adapatativo



Código Python — DWT

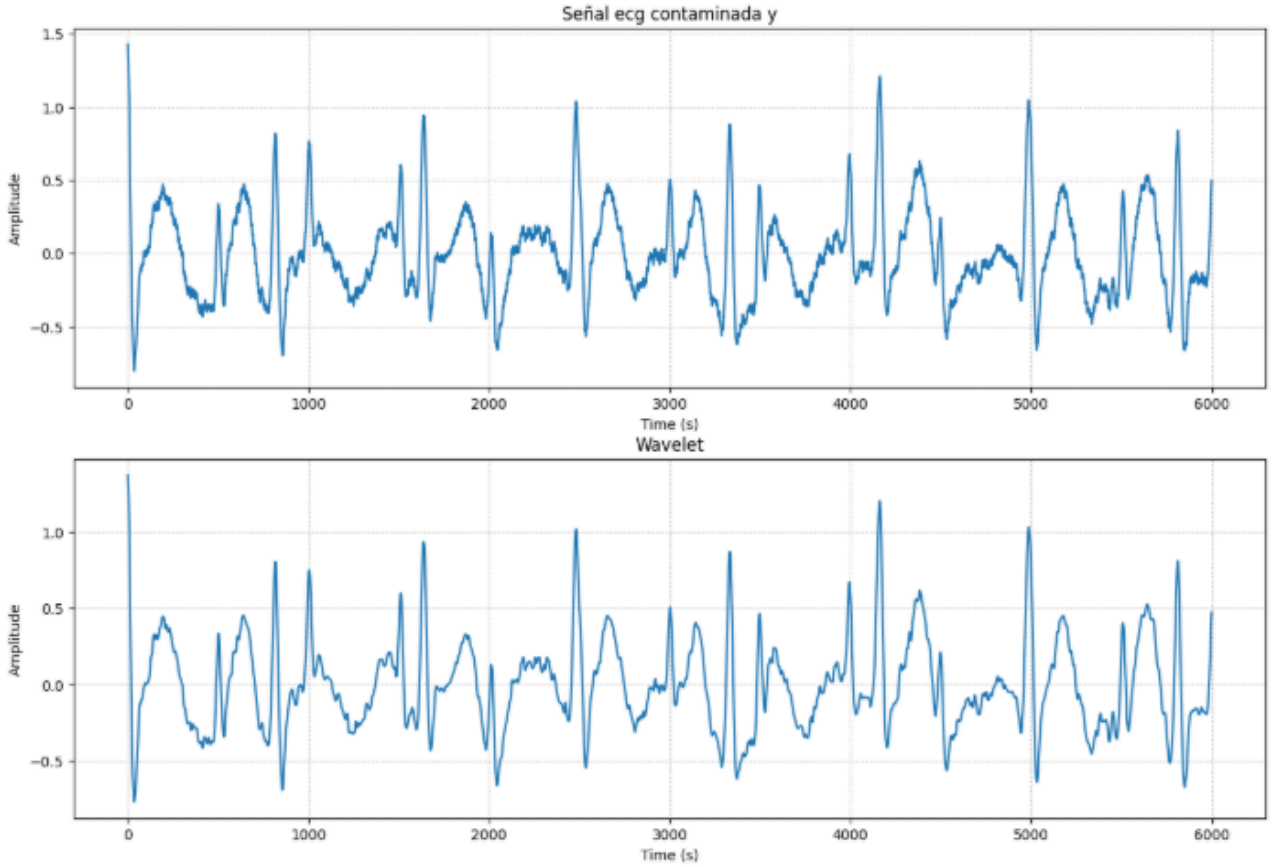
```python
import numpy as np
import pywt
import matplotlib.pyplot as plt

fs = 1000
t = np.linspace(0,1,fs)
x = np.sin(2*np.pi*10*t)

coeffs = pywt.wavedec(
    x, 'db4', level=4
)

coeffs[-1] = np.zeros_like(coeffs[-1])
coeffs[-2] = np.zeros_like(coeffs[-2])

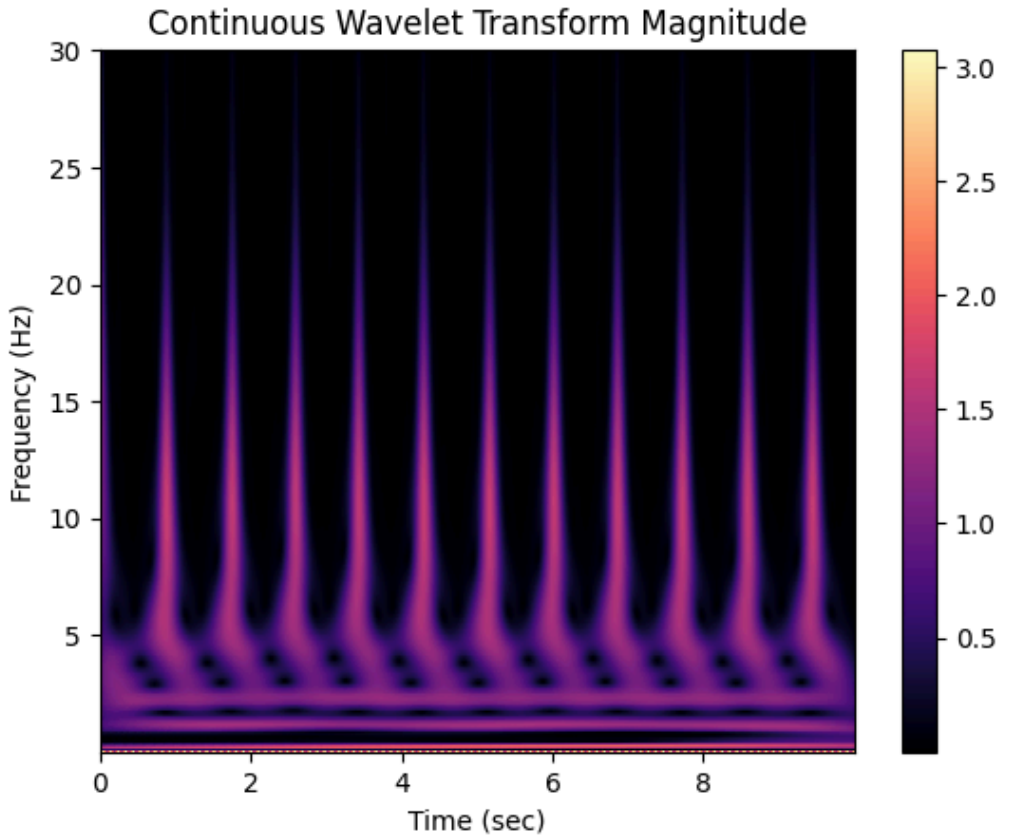
y = pywt.waverec(coeffs, 'db4')
```



Código Python — CWT

```
import neurokit2 as nk

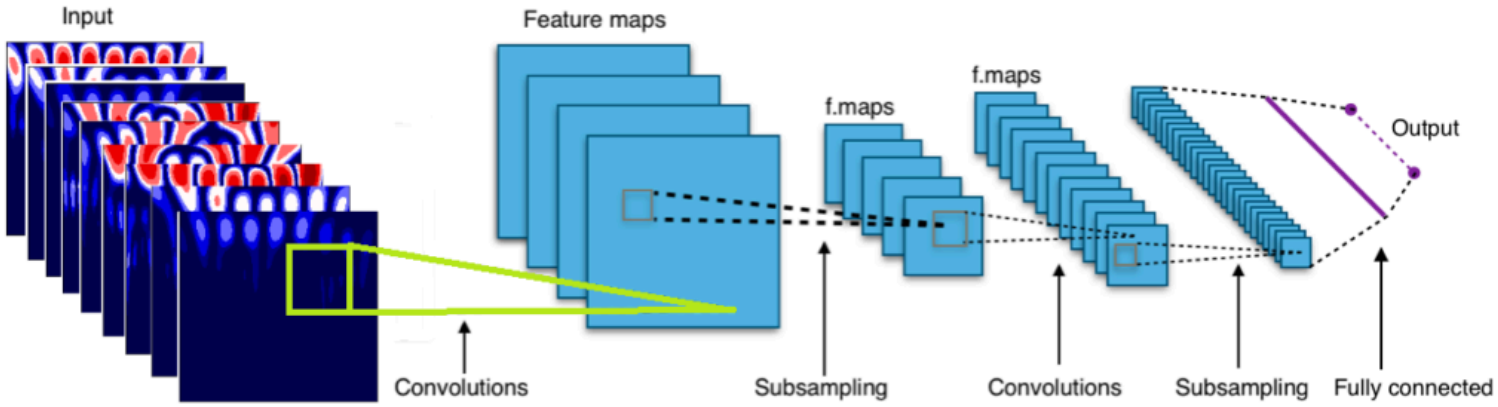
f, t, cwtm = nk.signal_timefrequency(
    ecg_signal,
    sampling_rate,
    max_frequency=30,
    method="cwt_cmor1.5-1.0",
    show=True
    )
```



Relación con Deep Learning

Actualmente existen enfoques híbridos:

- Wavelet + CNN
- Wavelet Scattering
- CNN sobre scalograms
- Features wavelet para ML



Las CNN aprenden filtros automáticamente

Super intro a filtros adaptativos (LMS)

Los filtros tradicionales poseen coeficientes fijos.

Cuando las características del ruido cambian con el tiempo, estos filtros pueden perder eficacia.

El LMS ajusta automáticamente sus coeficientes para minimizar el error entre una señal deseada y la salida del filtro.

$$e(n) = d(n) - y(n)$$

Ejemplo: **Cancelación Activa de Ruido (ANC)**

Los auriculares modernos utilizan filtros adaptativos para aprender el ruido ambiental y cancelarlo en tiempo real.



Partes de un filtros adaptativo LMS

- Filtro FIR adaptativo
- Error e(n)
- Algoritmo de actualización

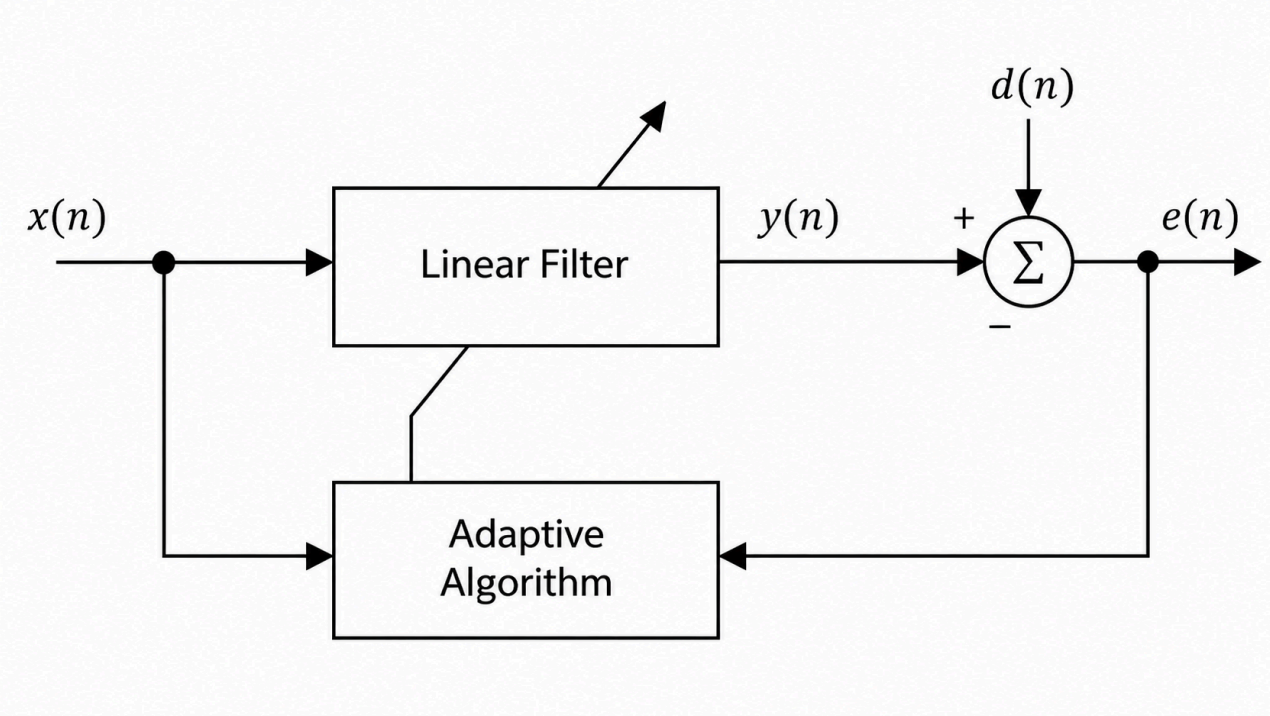
$$e(n) = d(n) - y(n)$$

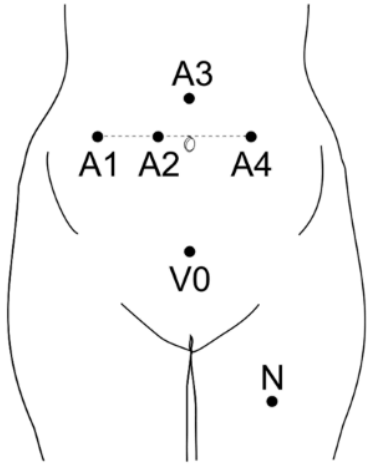
$$w(n+1) = w(n) + \mu e(n) x(n)$$

donde:

- $w(n)$ = coeficientes actuales
- μ = velocidad de aprendizaje
- $x(n)$ = ruido de referencia
- $d(n)$ = señal deseada

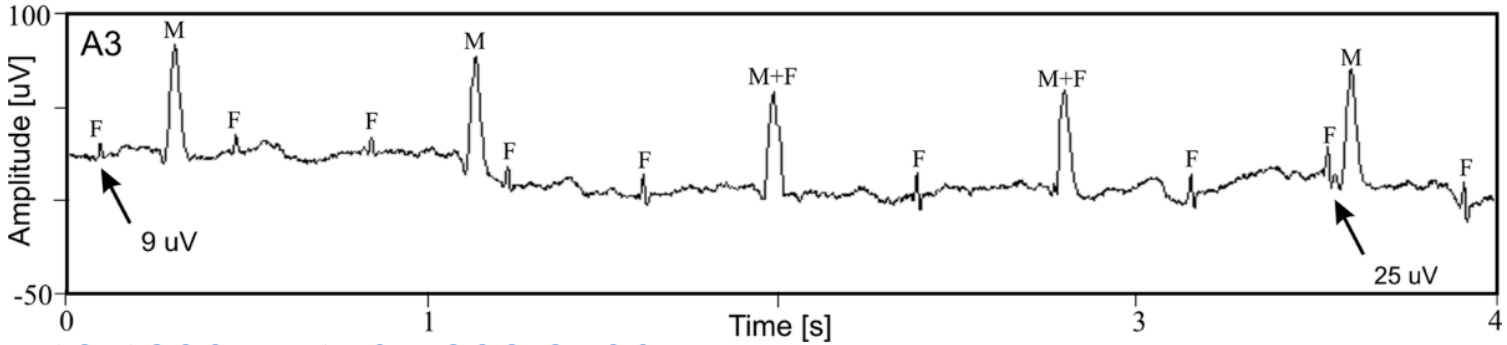
d(n)=ECG(n)+x(n)



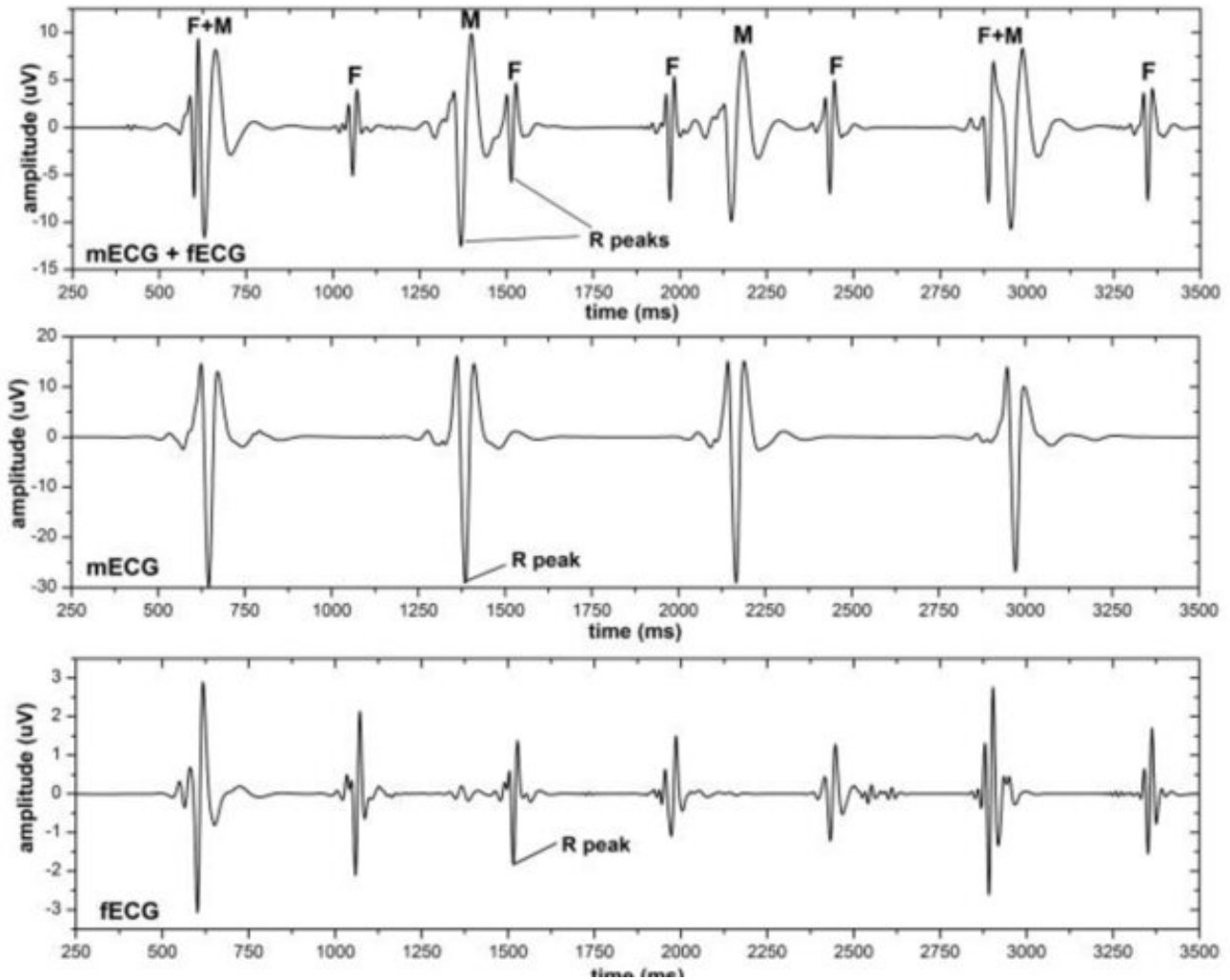


A1... A4 - abdominal leads
V0 - reference electrode
N - active ground





DOI: https://doi.org/10.1038/s41597-020-0538-z



DOI: 10.1109/EHB.2015.7391357

GRACIAS TOTALES!

Nooo, vamos al código!

link: https://drive.google.com/drive/folders/1pa2yOZy0g54EY9FwUdlhxtZg4HuB0WFZ?usp=sharing